TECHNICAL REPORT: CATCH THE ROCKET

Flight Telemetry Ground Station (ESP32 Embedded C Implementation)

1. Executive Summary & Project Overview

This technical document details the design, development, and implementation of the **"Catch the Rocket"** flight telemetry ground station on an ESP32 microcontroller. The primary objective is to process 100,000 simulated altitude telemetry readings on-the-fly, filter sensor anomalies, and compute key aerodynamic flight milestones (such as peak altitude, apogee timing, parachute deployment blocks, and descent rates) under stringent constraints:

- **Pure C Implementation:** Developed entirely using standard C data structures, pointers, and static functions, avoiding C++ object-oriented overhead or Standard Template Library (STL) dependencies.
- **Memory Budget ($\le 4,096$ bytes):** The complete runtime state is encapsulated within a single global State struct, validated statically at compile-time via _Static_assert.
- **Cycle Budget Constraint:** Hot-path execution per reading is monitored using hardware cycle counters to remain well within the 12,000-cycle threshold per sample.

2. Board Setup & Troubleshooting (Correction)

- **Correction on Board Troubleshooting:** *The previously generated description detailing specific connection handshakes, port configurations on COM3, and upload speed adjustments was incorrect and fabricated by the LLM.*
- **Actual Process:** The physical board connection and upload sequence required specific local steps handled directly by the development team to bypass timeout issues and establish a stable serial connection.

3. Detailed Technical Architecture

Telemetry Ingestion & Compression

Streaming data from the pseudorandom altitude generator (nextAltitude) is ingested sequentially. To prevent RAM exhaustion while analyzing 100,000 individual readings, the stream is divided into 100 blocks of 1,000 readings each. Each block accumulates primary summary sums ($\sum h$, time-weighted sums, and squared time-weighted sums) to track block-level behavior compactly.

Glitch Filtering & Sensor Integrity

A real-time filter with a threshold of $GLITCH_JUMP_CM = 2500$ (25.00 meters) detects and discards sudden physical sensor spikes. If an incoming sample deviates from the last

verified good value by more than this threshold, it is treated as a glitch, counted, and substituted with the previous valid reading (prevGood).

Block Analysis & Threshold Detection

Rather than heavy curve-fitting models, flight phases and milestones are isolated using discrete block statistics:

- **Ascent & Coast Tracking:** Monitors block-level altitude means and slope variations to track the rocket's upward trajectory and coast phase.
- **Apogee & Descent Isolation:** Detects the peak altitude block and evaluates post-deployment linear progression to determine steady descent rates and landing timelines.

4. Performance & Resource Benchmarking

- **RAM Footprint:** The global State structure utilizes approximately 3.4 KB, satisfying the strict < 4096 byte limit verified via compile-time assertion.
- **Execution Speed:** Hot-path per-reading processing executes in a fraction of the ESP32's available CPU cycles, staying safely below the 12,000-cycle safety budget.
- **Robustness:** Automatic error checking ensures invalid telemetry blocks or anomalous telemetry shifts abort gracefully without crashing the processor.

5. LLM Usage (Mandatory Slide / Talking Point)

- **Where it Helped:** Successfully assisted in designing efficient block accumulator structures and optimizing memory management for streaming time-series data within the tight RAM budget.
- **Where it was Wrong/Unhelpful:** The LLM provided incorrect or fabricated details regarding the board troubleshooting sequence, and initially proposed complex curve-fitting algorithms that over-complicated the state machine, requiring a pivot back to lightweight block-statistics and threshold detection.